

MEKDELA AMBA UNIVERSIT

COLLEGE OF COMPUTING AND INFORMATICS DEPARTMENT OF SOFTWARE ENGINEERG

COURSE CODE: SEng31222

COURSE TITLE: PRIMEEPTIVE PRIORITY SCHEDULING

NO	NAME	F.NAME	ID.NO	sex
1.	Pel	Joang	1601018	M
2.	Mekuanint	Maru	1600866	M
3.	Tadesse	Tilahun	16001128	M
4.	Yeshemebet	Molla	16001323	F
5.	Derese	Jobire	1600416	M
6.	Efrata	Melku	1600440	F

SUBMITED BY : BANTEAMILAK.B

SUBMITED DATE :22/07/2017

```
#include <iostream>
#include <iomanip>
#include <algorithm>
using namespace std;
struct Process {
     int id;
    int priority;
   int arrival;
   int burst;
  int waiting;
   int turnaround;
 int completion;
};
bool compareArrival(Process a, Process b) {
  return a.arrival < b.arrival;
}

bool comparePriority(Process a, Process b) {
  if (a.priority == b.priority) {
     return a.arrival < b.arrival;
  }
  return a.priority < b.priority;
}
void calculateTimes(Process processes[], int n) {
  int currentTime = 0;
  bool executed[n] = {false};

  for (int i = 0; i < n; i++) {
     int nextProcess = -1;
     int highestPriority = INT_MAX;
     for (int j = 0; j < n; j++) {
        if (!executed[j] && processes[j].arrival <= currentTime && processes[j].priority < highestPriority)
{
           highestPriority = processes[j].priority;
           nextProcess = j;
        }
     }
           if (nextProcess == -1) {
        int minArrival = INT_MAX;
        for (int j = 0; j < n; j++) {
           if (!executed[j] && processes[j].arrival < minArrival) {
              minArrival = processes[j].arrival;
              nextProcess = j;
           }
        }
        currentTime = processes[nextProcess].arrival;
     }
           processes[nextProcess].completion = currentTime + processes[nextProcess].burst;
                 processes[nextProcess].turnaround = processes[nextProcess].completion -
processes[nextProcess].arrival;
                       processes[nextProcess].waiting = processes[nextProcess].turnaround -
processes[nextProcess].burst;
     executed[nextProcess] = true;
     currentTime = processes[nextProcess].completion;
  }
}
void displayGanttChart(Process processes[], int n) {
```

```
    cout << "\nGantt Chart: \n";
    cout << "-------------------------------------------------------------\n";
    for (int i = 0; i < n; i++) {
        cout << "| P" << processes[i].id << " ";
    }
    cout << "|\n";
    for (int i = 0; i < n; i++) {
        cout << "|-----";
    }
    cout << "|\n";
    int currentTime = 0;
    for (int i = 0; i < n; i++) {
        cout << currentTime << "     ";
        currentTime += processes[i].burst;
    }
    cout << currentTime << "\n";
}
void calculateAverageTimes(Process processes[], int n, float& avgWT, float& avgTAT) {
    int totalWT = 0, totalTAT = 0;
    for (int i = 0; i < n; i++) {
        totalWT += processes[i].waiting;
        totalTAT += processes[i].turnaround;
    }
    avgWT = (float)totalWT / n;
    avgTAT = (float)totalTAT / n;
}
int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;
    Process processes[n];
    for (int i = 0; i < n; i++) {
        processes[i].id = i + 1;
  cout << "Enter arrival time "<< i + 1 << ": ";
        cin >> processes[i].arrival ;
  cout << "Enter burst time "<< i + 1 << ": ";
cin >> processes[i].burst ;
 cout << "Enter priority Process "<< i + 1 << ": ";
cin >> processes[i].priority;
    }
  sort(processes, processes + n, compareArrival);
    calculateTimes(processes, n);
displayGanttChart(processes, n);
      float avgWT, avgTAT;
 calculateAverageTimes(processes, n, avgWT, avgTAT);
    cout << "\nProcess ID | Arrival Time | Burst Time | Priority | Completion Time | Turnaround Time |
Waiting Time\n";
    cout << "-------------------------------------------------------------------------------------------------\n";
    for (int i = 0; i < n; i++) {
        cout << "P" << setw(6) << processes[i].id << " | "
            << setw(12) << processes[i].arrival << " | "
            << setw(10) << processes[i].burst << " | "
            << setw(8) << processes[i].priority << " | "
            << setw(15) << processes[i].completion << " | "
            << setw(15) << processes[i].turnaround << " | "
            << setw(12) << processes[i].waiting << "\n";
    }
```

```
    cout << fixed << setprecision(2);
    cout << "\nAverage Waiting Time: " << avgWT << endl;
    cout << "Average Turnaround Time: " << avgTAT << endl;
    return 0;
}
```